

UNIVERSITY OF DHAKA

**RedGrid:
Dependency-Constrained UCB
Exploration for Autonomous
Penetration Testing**

Submitted by

Class Roll: 50028

Registration No: H-55

Class Roll: 50040

Registration No: H-49

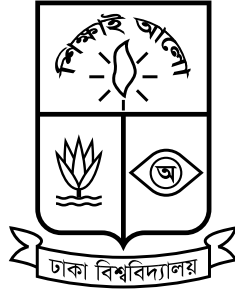
Submitted to the

Department of Computer Science and Engineering

University of Dhaka, Bangladesh

in partial fulfillment of the requirements for the degree of
Professional Masters in Information and Cyber Security (PMICS).

Declaration Form



University of Dhaka

We hereby declare that the work in this report titled “**RedGrid: Dependency-Constrained UCB Exploration for Autonomous Penetration Testing**” has been carried out by us under the supervision of Dr. Md. Abdur Razzaque. We further declare that where information has been derived from the literature, we have adequately cited and referenced the original sources. No part of this report was previously presented for another degree or diploma at this or any other institution.

Name of the Candidate

Nishan Paul

Name of the Candidate

Md Rakibur Rahman

Dr. Md. Abdur Razzaque

Professor and Chairman

Dept. of

Computer Science and Engg.

University of Dhaka

“To conquer fear, you must become fear.”

Ra’s al Ghul, Batman Begins

UNIVERSITY OF DHAKA

Abstract

Autonomous penetration testing agents powered by large language models have shown rapid progress in recent years, yet published evaluations consistently point to two unresolved weaknesses: agents explore candidate vulnerabilities too narrowly, and they struggle to reason about the prerequisite relationships between different stages of an attack. These weaknesses limit how far current systems can be trusted to operate against realistic, multi-step web, API, and network targets without close human supervision.

This thesis is an early-stage investigation into whether explicitly modeling the dependencies between attack steps – rather than treating candidate vulnerabilities as an unordered list – can improve how autonomous agents explore and prioritize their actions during a penetration test. As a first step, we conducted a systematic review of the recent literature on LLM-based offensive security agents, spanning web application exploitation, GraphQL API testing, and multi-host network attacks, to identify recurring architectural patterns, common failure modes, and the benchmarks used to evaluate such systems. Building on this review, we have begun drafting an initial system architecture intended to structure how an agent accumulates knowledge about a target and decides which attack path to pursue next.

At this stage, our contribution is preliminary: a consolidated view of the current state of autonomous penetration testing research and an early architectural direction motivated by the gaps observed in it. The remaining work over the coming months will focus on refining this architecture, implementing it, and evaluating it against established benchmarks to determine whether dependency-aware exploitation meaningfully improves exploitation performance over existing approaches.

Acknowledgements

We are sincerely grateful to our supervisor, Dr. Md. Abdur Razzaque, Professor and Chairman, Department of Computer Science and Engineering, University of Dhaka, for agreeing to guide this work and for the time he has already invested in it at such an early stage. His feedback pushed us to move beyond simply summarizing existing autonomous penetration testing systems and to instead ask what specific gap in that body of work our own research could realistically address. That question shaped the direction this thesis has taken so far, and we expect it to keep shaping the work as it progresses over the coming months.

We would also like to acknowledge the wider research community whose published work – spanning benchmark design, agent architectures, and empirical evaluations in offensive security – gave us the foundation on which this thesis is being built. Any progress we make over the next six months will rest heavily on the groundwork laid by that literature.

Contents

Declaration Form	i
Abstract	iii
Acknowledgements	iv
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives	4
1.4 Scope of the Project	5
1.5 Anticipated Contributions	6
1.6 Anticipated Challenges	7
1.7 Organization	9
2 Literature Review and Related Work	10
2.1 Domain Overview	10
2.2 Existing Systems	11
2.2.1 Single-Agent and ReAct-Style Approaches	11
2.2.2 Multi-Agent and Team-Based Orchestration	12
2.2.3 Planning-Augmented Agents	12
2.2.4 Multi-Host and Network-Scale Red Teaming	13
2.2.5 API and GraphQL-Specific Testing	13
2.2.6 Adaptive and Computer-Use Web Testing	13
2.2.7 Memory, Reflection, and Skill Accumulation	14
2.2.8 Benchmarks and Evaluation Frameworks	14
2.3 Comparative Analysis	15
2.4 Research Gap	16
2.5 Summary	17
3 Proposed Methodologies	19
3.1 System Overview	19

3.2	Requirement Analysis	19
3.3	Proposed Architecture	19
3.4	System Workflow	19
3.5	Tools and Technologies	19
4	Execution Plan	20
4.1	Work Breakdown Structure	20
4.2	Project Timeline	20
4.3	Milestones	20
4.4	Resource Requirements	20
4.5	Risk Assessment	20
5	Experimental Results	21
5.1	Evaluation Plan	21
6	Conclusions	22
6.1	Summary	22
6.2	Expected Deliverables	22
6.3	Future Works	22
	Bibliography	23
	Appendix A: RedGrid Reference Material	27
.1	Full VDG Node Schema	27
.2	Ordinal Evidence Score (E_{ord}) Reference	29
.3	Benchmark Suite Summary	29
.4	UCB Hyperparameter Search Grid	30
.5	Ablation Condition A1 — Precise Pseudocode	31
	Condition (b) — UCB with dependency-constrained eligible set (pre-filter):	31
	Condition (c) — Stacked: UCB over all nodes, post- filter by dependency (the key discriminant):	31
.6	Prior Work Gap Summary	32

List of Figures

1.1	Exploration-failure rate as the dominant CVE-Bench failure mode, by agent and mode (data from CVE-Bench’s Table 5 failure-mode breakdown [1]).	3
-----	--	---

List of Tables

1.1	PentestEval ground-truth-injection ablation: marginal gain from replacing each stage’s output with expert ground truth, applied cumulatively on top of the SMP baseline [2].	3
1.2	Attack surfaces we are currently considering in scope, and their candidate governing benchmarks.	6
2.1	Comparison of representative autonomous VAPT systems surveyed	15
1	Complete VDG Node Schema	27
2	Ordinal Evidence Score (E_{ord}) Rubric	29
3	Full REDGRID Benchmark Suite	30
4	UCB Hyperparameter Search Grid	31
5	Prior Work Gaps and REDGRID Fixes	32

List of Algorithms

Acronym	What (it) Stands For
LAH	List Abbreviations Here

Constant Name	Symbol	=	Constant Value (with units)
Speed of Light	c	=	$2.997\,924\,58 \times 10^8 \text{ ms}^{-\text{s}}$ (exact)

symbol	name	unit
a	distance	m
P	power	W (Js ⁻¹)
ω	angular frequency	rads ⁻¹

For/Dedicated to/To my...

Chapter 1

Introduction

1.1 Background and Motivation

Penetration testing – the authorized, systematic attempt to discover and exploit weaknesses in a target system before a real adversary does – has traditionally resisted automation because it demands the kind of open-ended, context-sensitive reasoning that scripted scanners cannot perform. Over the past two years, agentic Large Language Models (LLMs) have repeatedly closed parts of this gap. Fang et al. [3] showed that a single LLM agent equipped with a browser and a small set of tools could autonomously perform multi-step SQL-injection attacks against live websites without being told which vulnerability to look for. Fang et al. [4] extended this to real, critical-severity CVEs, showing that GPT-4 could exploit 87% of a 15-vulnerability benchmark when given the CVE description, though this collapsed to 7% once the description was withheld – early evidence that *exploiting* a known vulnerability and *discovering* one are almost entirely different capabilities. Zhu et al. [5] pushed further into genuinely zero-day territory with a hierarchical team of planning and task-specific agents (HPTSA), and a wide subsequent literature has since proposed increasingly elaborate multi-agent, memory-augmented, and planning-augmented architectures for autonomous vulnerability assessment and penetration testing (VAPT).

Our starting point for this thesis was to understand this literature well enough to know whether it had already solved the problem, and if not, where exactly it had not. Over the past several weeks we assembled and read a corpus of 29 papers spanning single-agent exploitation studies, hierarchical multi-agent frameworks, classical-planning hybrids, cross-mission memory systems, and the major benchmark suites used to evaluate them (surveyed in full in Chapter 2). One finding recurs across at least six independently developed systems in this corpus (AWE, AutoPT, VulnBot, PentestAgent, D-CIPHER, and Incalmo): **architecture, not model scale, appears to be the dominant variable in autonomous exploitation performance** – a well-structured pipeline running a comparatively inexpensive model tends to outperform an unstructured, loosely-scaffolded agent loop running a frontier model. At the same time, the same reading surfaced two failure modes that keep showing up independently across papers and that no single surveyed system appears to address together. Those two gaps, described below, are what this thesis has chosen to investigate first. We want to be upfront that at this stage this is a direction we are pursuing and testing, not a result we have already obtained.

1.2 Problem Statement

Failure Mode 1 – Insufficient exploration. On CVE-Bench [1], a benchmark of 40 critical ($CVSS \geq 9.0$) real-world web-application CVEs, the strongest reported agent exploits only 13% of one-day and 10% of zero-day vulnerabilities. CVE-Bench’s own failure-mode breakdown identifies *insufficient exploration* as the dominant cause of failure across every agent and evaluation setting, with exploration-failure rates ranging from 37.5% to 80.0% (T-Agent: 80.0% zero-day / 55.0% one-day; AutoGPT: 72.5% / 45.0%; Cy-Agent: 67.5% / 37.5%), as illustrated in Figure 1.1. Crucially, this does not appear to be a reasoning-quality problem so much as a breadth-of-search problem: agents commit early to a narrow set of candidate attack paths and rarely return to explore the rest of the surface.

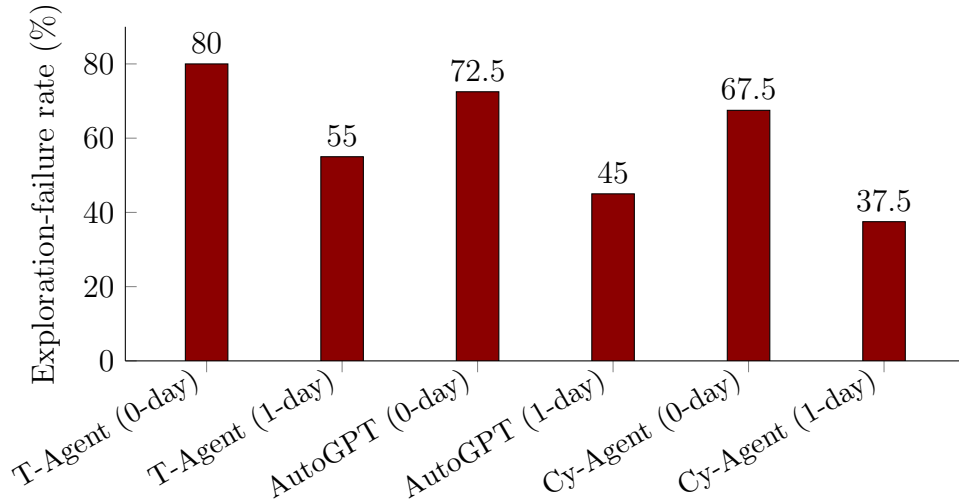


FIGURE 1.1: Exploration-failure rate as the dominant CVE-Bench failure mode, by agent and mode (data from CVE-Bench’s Table 5 failure-mode breakdown [1]).

Failure Mode 2 – The dependency-reasoning gap. PentestEval [2] decomposes the pentesting pipeline into six stages – Information Collection (IC), Weakness Gathering (WG), Weakness Filtering (WF), Attack Decision-Making (ADM), Exploit Generation (EG), and Exploit Revision (ER) – and finds the opposite bottleneck at the *planning* level: ADM, which requires reasoning about prerequisite dependencies between candidate weaknesses, is the single lowest-scoring stage (Spearman correlation 0.25 against expert judgement). PentestEval’s ground-truth-injection ablation quantifies roughly how much room for improvement exists at each stage, summarized in Table 1.1.

TABLE 1.1: PentestEval ground-truth-injection ablation: marginal gain from replacing each stage’s output with expert ground truth, applied cumulatively on top of the SMP baseline [2].

Configuration	End-to-end success	Marginal gain
SMP (baseline)	0.31	–
+ GT Weakness Gathering (WG)	0.50	+0.19
+ GT Weakness Filtering (WF)	0.53	+0.03
+ GT Attack Decision-Making (ADM)	0.67	+0.14

ADM delivers the largest single-stage marginal increment (+0.14) of the three tested – measured, notably, on top of an already ground-truthed WG+WF pipeline, not in isolation. Any dependency structure an agent grows on its own, without ground

truth, is necessarily a weaker approximation than this, so a realistic ceiling for any such approach would have to sit below 0.67 rather than above it.

The gap we have not found closed in the literature. Systems that target exploration breadth (T-Agent, HPTSA [5], MAPTA [6]) tend to dispatch flat, undifferentiated task lists with no explicit prerequisite or dependency model. Systems that target dependency reasoning (PentestEval’s SMP baseline, CHECKMATE-style classical-planning approaches [7]) are evaluated on curated scenarios with pre-enumerated weakness sets and do not appear to scale to open-ended, wide-surface exploration. Within the 29 papers we surveyed, we did not find a system that combines exploration over a dependency-aware structure, prerequisite relationships grown dynamically from live discovery during a mission, and evaluation across more than one independently-benchmarked attack-surface family. Investigating whether closing this compound gap is possible, and whether doing so is actually worthwhile, is the problem this thesis sets out to explore.

1.3 Objectives

Guided by the two gaps identified in Section 1.2, we have set out the following objectives for this thesis. These are the goals we are working toward over the coming months, not a checklist of work already completed; the exact design of each component is expected to change as our implementation and early experiments give us feedback that the literature alone cannot.

1. To investigate whether representing candidate vulnerabilities as a graph with explicit prerequisite relationships, rather than as an unordered list, can meaningfully improve how an autonomous agent chooses what to explore next – and to design and formalize such a structure, which we are currently calling a **Vulnerability Dependency Graph (VDG)**, to the point where it can be implemented and tested.

2. To explore ways of separating what an agent has *confirmed* about a target from what it is merely *hypothesizing*, so that the quality of discovery and the quality of planning can eventually be evaluated somewhat independently of each other.
3. To settle on a multi-layer orchestration structure for the overall agent – drawing on the structural patterns we observed recurring across the strongest-performing systems in our survey – and to work out at what level of detail this structure needs to be specified before it is implementable.
4. To examine whether letting an agent retain and reuse strategies across multiple missions offers a genuine advantage, or whether it mainly introduces risk (e.g., a strategy that worked against one version of a technology being harmful against another).
5. To identify which existing, published benchmarks are suitable for evaluating this kind of system, and to begin drafting an evaluation plan around them rather than constructing new benchmarks of our own.

1.4 Scope of the Project

Based on our reading of the literature so far, we intend to bound this project by one governing rule: we plan to work only with attack surfaces for which a dedicated, reusable, oracle-backed benchmark already exists in the surveyed literature, rather than constructing new benchmarks ourselves. This is a scope we have provisionally settled on; it may narrow further once early implementation work tells us more about what is realistically achievable within the remaining timeline. Table 1.2 summarizes the attack-surface families we are currently considering in scope and the benchmarks that would bound any claim made about them.

What we currently intend to leave out. General REST API exploitation is not something we plan to evaluate directly: RESTler’s [14] evaluation targets are one-off real-world case studies rather than a standardized, reusable benchmark, so we did

TABLE 1.2: Attack surfaces we are currently considering in scope, and their candidate governing benchmarks.

Attack surface	Benchmark(s)	Representative coverage
Web application (HTTP/HTML)	Fang et al. 15-vuln sandbox [4]; HPTSA 14-CVE zero-day suite [5]; CVE-Bench (40 CVEs) [1]; MAPTA/XBOW (104) [6]; Hack-World (36) [8]; PentestEval (346 tasks) [2]; Cybench [9]; PentestGPT/HTB machine sets [10]	SQLi, XSS, CSRF, SSRF, SSTI, LFI, file-upload RCE, IDOR, auth bypass, framework RCEs, JWT forgery
GraphQL APIs	PrediQL 6-API suite [11]	Schema abuse, dependency-chain injection, batched-auth bypass, IDOR, nested-query DoS
Multi-host / Active Directory	Incalmo MHBench (40 environments) [12]	Lateral movement, credential reuse, privilege escalation
Production system corpus (hard tier)	BountyBench (25 real systems) [13]	Economic/adversarial validation layer over the web surface

not find an equivalent of the other benchmarks above for REST APIs specifically. Some of RESTler’s underlying techniques (dependency inference between API calls, response-feedback pruning) may still turn out to be useful internally, e.g. within GraphQL testing, but we do not plan to report REST-specific results. For the same reason – the absence of a suitable existing benchmark within our survey – binary exploitation, physical- and network-layer attacks, and social engineering are also outside what we currently intend to attempt.

1.5 Anticipated Contributions

We want to be careful not to overstate what this thesis has established at this point – none of the following has been implemented or evaluated yet. What follows is the direction we currently believe is worth pursuing, framed as hypotheses to be tested rather than results already in hand. We expect this list to be revised, narrowed, or partly abandoned as our implementation and early experiments give us evidence the literature review alone cannot.

- **Dependency-aware exploration.** Our primary hypothesis is that combining exploration-style candidate selection with an explicit, dynamically-grown model of prerequisite relationships between vulnerabilities – rather than treating either concern in isolation – can improve how thoroughly and how sensibly an autonomous agent searches a target. We intend to test this against CVE-Bench and PentestEval once an implementation exists, but we are not yet in a position to state expected numbers with confidence.
- **Reuse of prior experience across missions.** We would like to examine whether allowing an agent to retain and selectively reuse strategies from earlier engagements produces a measurable benefit on targets that share underlying technology, without introducing the negative-transfer risk described in Section 1.6. Whether this ends up being a core contribution or supporting infrastructure will depend on what our early ablations show.
- **An honest, cross-benchmark evaluation.** Independently of whether the above hypotheses hold, we intend for the evaluation itself – run consistently across more than one existing, independently-published benchmark, with standardized oracles and clearly separated per-surface reporting – to be a useful methodological contribution to how this class of system gets evaluated, since we did not find this done consistently in the papers we surveyed.

Several further mechanisms that recur across the surveyed literature – structured hand-off between agent components, retry-and-diagnose validation loops, and detailed run logging – currently look like necessary supporting infrastructure for any implementation of the above rather than contributions in themselves. We plan to revisit this framing once implementation is further along.

1.6 Anticipated Challenges

Several challenges surfaced repeatedly while we were reading the literature, and we expect them to shape our work over the next six months regardless of which

specific design choices we end up making. We are listing them here, early, rather than waiting to discover them the hard way during implementation:

1. **Inferring dependencies without ground truth.** Any prerequisite structure our agent builds on its own will be inferred by an LLM rather than annotated by an expert, and we expect this to be the single riskiest part of whatever we build. Before relying on such a structure for any evaluation, we plan to first run a small pilot study against PentestEval’s expert-annotated dependencies to get a sense of how accurate our inference actually is.
2. **Sandbox results may not transfer to the real world.** Fang et al. [3] found only 1 exploitable XSS in 50 real-world candidate sites (2%), against 73.3% in a matched sandbox. We intend to keep this gap in mind throughout and to avoid presenting sandbox numbers as if they were real-world numbers.
3. **Some benchmarks are small.** PrediQL’s GraphQL suite, for instance, covers only 6 APIs – real and standardized, but far smaller than benchmarks like CVE-Bench or XBOW. Any conclusion we draw from it will need to be stated as exploratory rather than definitive.
4. **Cost is a moving target.** Whatever cost-per-exploit figures we eventually report will be tied to a specific model and pricing at a specific point in time, and we plan to state that explicitly rather than presenting cost as a fixed property of the approach.
5. **Reused strategies could backfire.** A strategy that worked against one version of a piece of software could plausibly be actively harmful against a different version. We have not found this risk addressed in the 29 papers we surveyed, which is one reason we consider it worth paying attention to from the start.
6. **Tuning could be fragile.** If our approach ends up depending on several tunable parameters, we would want to check how sensitive results are to those choices rather than reporting numbers from a single, possibly lucky, configuration.

7. **Being honest about what generalizes.** If we do end up testing across more than one attack surface, we want to be careful about the difference between "one shared design" and "one literally unmodified system," since we expect surface-specific components to be necessary either way.

1.7 Organization

This report reflects roughly the first month of work on this thesis, and its organization is intended as a plan for the full document we expect to submit in approximately six months, not a description of chapters that already exist in complete form. Chapter 2 presents our survey of the 29-paper corpus underlying this work, organized by architectural pattern, and the gaps in prior work that motivated the direction described in Sections 1.2 and 1.3; this chapter is the most complete at present. Chapter 3 will present our system design as it develops – currently sketched at a high level, but expected to be substantially revised as implementation proceeds. Chapter 4 will lay out our work breakdown, timeline, and milestones for the remaining months, and will be kept up to date as work progresses. Chapter 5 will specify and, eventually, report on our evaluation plan once an implementation exists to evaluate. Chapter 6 will summarize the completed work, and is not expected to be written in any substantive form until much later in the thesis timeline.

Chapter 2

Literature Review and Related Work

2.1 Domain Overview

Autonomous penetration testing – the use of Large Language Model (LLM) agents to independently reconnoitre, identify, and exploit weaknesses in a target system without step-by-step human direction – has emerged as an active research area only in the last two to three years. The domain sits at the intersection of two older fields: Vulnerability Assessment and Penetration Testing (VAPT), which traditionally relies on human operators following structured methodologies (reconnaissance, weakness identification, exploitation, and reporting), and LLM agent research, which studies how language models can plan, use tools, and act over multiple steps toward a goal. RedGrid is situated in this intersection: it is concerned with how an LLM-driven agent, or a coordinated set of agents, can carry out the VAPT workflow autonomously against realistic, benchmarked targets.

A useful way to frame the domain is around the two capabilities an autonomous VAPT agent must exhibit simultaneously. The first is *exploration*: the agent must search a large space of candidate weaknesses across an unfamiliar attack surface without being told in advance which one is exploitable. The second is *reasoning*

about prerequisites: many real exploits are not single actions but chains, where one weakness (for example, an information disclosure or a misconfigured endpoint) must be established before a second, more consequential weakness becomes reachable. Recent benchmarking work suggests that these two capabilities are evaluated very differently in the literature, and that most existing systems have been built to strengthen one at the expense of the other – a pattern that motivates a closer reading of the existing systems and benchmarks surveyed in this chapter.

The chapter that follows first surveys the systems that have been proposed for this problem (Section 2.2), then draws out the recurring architectural choices across them (Section 2.3), before identifying, at a preliminary level, where the surveyed literature appears to leave a gap (Section 2.4). As this is an early-stage report, the discussion below reflects the literature review conducted so far rather than a finalized research direction; the survey will continue to be refined over the course of the thesis.

2.2 Existing Systems

This section surveys the systems, frameworks, and benchmarks identified during the literature review conducted so far. The review currently covers close to thirty papers; the discussion below groups them by the architectural approach they take, since this grouping turned out to be more informative than a chronological ordering.

2.2.1 Single-Agent and ReAct-Style Approaches

The earliest work in this space used a single LLM agent operating in a ReAct-style loop – observe, reason, act – against a target. Fang et al. showed that a single GPT-4 agent with tool access could autonomously exploit real-world one-day vulnerabilities when given the CVE description, in a pipeline reported to require under a hundred lines of code, while every other tested model and open-source scanner failed entirely on the same benchmark [4]. An earlier study by the same

group had already shown single agents hacking simplified toy websites [3], and Happe and Cito independently reported similar single-agent capability on privilege-escalation tasks [15]. PentestGPT extended the single-agent design with an explicit reasoning/generation/parsing split to mitigate context loss over long sessions [10], and a later paper by an overlapping set of authors distilled what design choices actually make single-agent systems effective in real-world settings [16].

2.2.2 Multi-Agent and Team-Based Orchestration

A second line of work replaces the single agent with a team of cooperating agents, motivated by the observation that a single flat loop struggles to scale to wide, unfamiliar attack surfaces. HPTSA introduced a hierarchical planner that dispatches task-specific sub-agents and was the first system reported to exploit real-world zero-day vulnerabilities autonomously [?]. MAPTA (also referred to as XBOW in the surveyed material) applies a similar team-based structure specifically to web application testing [?], while VulnBot [17] and PentestAgent [?] propose comparable multi-agent collaborative frameworks with role specialization (for example, separating reconnaissance, planning, and exploitation into distinct agent roles). Co-RedTeam extends this idea with an orchestration layer aimed at coordinating discovery and exploitation across agents [?]. These security-specific systems build on general-purpose multi-agent frameworks surveyed alongside them, notably AutoGen [18] and MetaGPT [?], which establish conversational and role-based multi-agent conventions without being security-specific themselves.

2.2.3 Planning-Augmented Agents

A smaller set of papers pairs LLM agents with a more classical planning component, aiming to give the agent an explicit model of prerequisites between actions rather than relying purely on the LLM’s implicit reasoning. Wang et al. combine LLM agents with classical planning techniques for penetration testing [?], and

D-CIPHER proposes a planner-plus-heterogeneous-executor architecture for offensive security tasks [19]. These systems report stronger performance on curated, pre-enumerated scenarios, which is discussed further in Section 2.3.

2.2.4 Multi-Host and Network-Scale Red Teaming

Beyond single web targets, two papers extend autonomous testing to multi-host and Active Directory-style network environments. Incalmo provides an autonomous LLM-assisted system for red-teaming multi-host networks, evaluated on a dedicated benchmark suite of forty environments [12]. Happe and Cito’s later work specifically studies whether LLM agents can perform assumed-breach penetration testing against enterprise Active Directory networks [20], reporting on lateral movement and credential-reuse behaviour that single-host web testing does not exercise.

2.2.5 API and GraphQL-Specific Testing

A separate strand of the literature targets API-level surfaces rather than the rendered web application. RESTler is an older, non-LLM stateful REST API fuzzer that infers producer-consumer dependencies between API requests from the API specification [14]; its dependency-inference technique is referenced by several of the more recent LLM-based papers as a useful building block rather than a competing end-to-end system. PrediQL applies LLMs specifically to GraphQL API testing, targeting introspection-driven schema abuse and similar GraphQL-specific weaknesses [11].

2.2.6 Adaptive and Computer-Use Web Testing

AWE proposes an adaptive agent architecture aimed at dynamic web applications [21], AutoPT frames end-to-end web penetration testing as a state-machine problem and asks explicitly how far current systems are from full automation [?], and HackWorld evaluates computer-use agents – agents that interact with a browser

the way a human would, via clicks and screenshots, rather than through raw HTTP calls – on web application exploitation [8].

2.2.7 Memory, Reflection, and Skill Accumulation

Several non-security agent papers are relevant background for how LLM agents retain and reuse experience across tasks. VOYAGER demonstrates an open-ended embodied agent that accumulates a persistent skill library across episodes [22], and Reflexion shows that verbal self-reflection on past failures can improve an agent’s subsequent attempts without any weight updates [23]. Neither paper is security-specific, but both are cited in the surveyed VAPT papers as the conceptual basis for cross-session memory mechanisms.

2.2.8 Benchmarks and Evaluation Frameworks

Finally, a distinct set of papers contributes benchmarks rather than agent architectures. CVE-Bench evaluates agents against forty critical real-world web-application CVEs with an oracle-backed pass/fail signal [1]. PentestEval decomposes penetration testing into discrete stages – weakness gathering, weakness filtering, and attack decision-making – and measures agent performance stage by stage rather than only end-to-end [2]. CYBENCH offers a broader capture-the-flag-style evaluation of cybersecurity capability across forty tasks [9], and BountyBench evaluates agents against twenty-five real production open-source systems using bug-bounty-style, dollar-denominated impact scoring [13]. Two survey papers were also reviewed for broader context: one surveys LLM-based agents in autonomous cyberattacks generally [?], and the other surveys LLM-based autonomous agents outside the security domain [?].

2.3 Comparative Analysis

Table 2.1 summarizes a representative subset of the surveyed systems along four dimensions that recurred throughout the literature review: whether the system uses a single agent or a coordinated team, whether it models dependencies between candidate weaknesses explicitly, whether it retains any form of memory across sessions, and which benchmark(s) it was evaluated against. The full survey covers more systems than shown here; the table is restricted to the systems most directly comparable to one another.

System	Architecture	Dependency Modeling	Cross-Session Memory	Benchmark(s)
Fang et al. [4]	Single agent (Re-Act)	None (given CVE description)	None	15 one-day CVEs
PentestGPT [10]	Single agent, split reasoning/-parsing	Implicit (LLM reasoning only)	None	HTB/VulnHub machines
HPTSA [?]	Hierarchical planner + task agents	None (flat dispatch)	None	15 zero-day CVEs
MAPTA [?]	Multi-agent team	None (flat dispatch)	None	CTF challenges
VulnBot [17]	Multi-agent, role-specialized	None (flat dispatch)	None	HTB-style scenarios
CHECKMATE [?]	Agent + classical planner	Explicit, pre-enumerated	None	Curated scenarios
PentestEval SMP [2]	Modular pipeline	Explicit, ground-truth-injected	None	12 real-world scenarios
Incalmo [12]	Multi-host orchestration	Partial (host/-credential graph)	None	MHBench (40 environments)
PrediQL [11]	LLM-guided fuzzer	Schema-derived	None	6 GraphQL APIs

TABLE 2.1: Comparison of representative autonomous VAPT systems surveyed

A few patterns are already visible from this comparison, even at this early stage of the review. First, systems that scale well to a wide, unfamiliar attack surface (HPTSA, MAPTA, VulnBot) tend to dispatch tasks flatly, without an explicit model of which weaknesses depend on which others. Second, systems that do model dependencies explicitly (CHECKMATE, PentestEval’s SMP configuration) tend to be evaluated on curated scenarios where the relevant weaknesses are already known in advance, rather than on open-ended discovery. Third, cross-session memory – the ability for an agent to retain and reuse what it learned in a previous session against a similar target – is largely absent from the systems surveyed so far, with the exception of ideas borrowed from non-security agent work such as VOYAGER [22] and Reflexion [23]. Finally, every system in the table is evaluated against a single benchmark family (web, GraphQL, or multi-host), and none of the surveyed papers report results across more than one attack-surface category using the same architecture.

These observations are preliminary and are discussed further, together with their implications, in the following section.

2.4 Research Gap

The literature surveyed in this chapter suggests, tentatively, that autonomous VAPT systems have so far been optimized for one of two capabilities at a time, rather than both together. On one side, systems built for broad exploration of an unfamiliar attack surface – HPTSA, MAPTA, and similar team-dispatch architectures – do not appear to model prerequisite relationships between candidate weaknesses explicitly; they search widely but without a formal notion of what must be established before what. On the other side, systems that do reason explicitly about such dependencies, including CHECKMATE’s classical-planning layer and PentestEval’s ground-truth attack decision-making configuration, are evaluated on scenarios where the relevant weaknesses have already been enumerated in advance, and it is not yet clear from

the surveyed papers how well these approaches would scale to open-ended discovery on a wide, previously unseen surface.

This split is echoed in how the two most detailed benchmarks in the corpus report their results. CVE-Bench’s own failure analysis attributes the majority of unsuccessful attempts on its hardest real-world CVEs to insufficient exploration rather than to reasoning quality, while PentestEval’s stage-level breakdown finds that the attack-decision-making stage – reasoning about which weakness to pursue next given what has already been discovered – is consistently the weakest stage across the systems it evaluates. Taken together, these two findings point at what looks, at this preliminary stage of the review, like two sides of the same underlying gap: no system surveyed so far appears to combine broad, open-ended exploration with an explicit, dynamically-constructed model of dependencies between discovered weaknesses.

A secondary observation, of lower priority but still worth noting, is that cross-session memory and evaluation across more than one attack-surface family (web, GraphQL, multi-host) are both rare in the surveyed corpus, which may also represent open areas worth further reading.

It should be stressed that this is an early-stage reading of the literature. The gap described above is offered as a working hypothesis to guide the remainder of the review and the early planning of the thesis, rather than as a finalized problem statement or a commitment to a specific architectural solution – both of which will be developed and justified more carefully as the work progresses over the coming months.

2.5 Summary

This chapter has surveyed close to thirty papers relevant to autonomous, LLM-driven penetration testing, covering single-agent systems, multi-agent orchestration, planning-augmented agents, multi-host red teaming, API/GraphQL-specific testing, and the benchmark datasets used to evaluate them. A comparative reading of this

literature points, tentatively, to a recurring split between systems optimized for broad exploration and systems optimized for dependency-aware reasoning, with few if any surveyed systems combining both, and few evaluated across more than one attack-surface family.

As this report reflects an early stage of the thesis, the survey above should be read as a snapshot of work completed so far rather than a closed literature review. Over the coming months, the review will be deepened – in particular around the planning-augmented and memory-augmented systems, where the fewest papers were found – and the observations in Section 2.4 will be used to motivate a more precisely scoped research question and methodology in the chapters that follow.

Chapter 3

Proposed Methodologies

3.1 System Overview

3.2 Requirement Analysis

3.3 Proposed Architecture

3.4 System Workflow

3.5 Tools and Technologies

Chapter 4

Execution Plan

4.1 Work Breakdown Structure

4.2 Project Timeline

4.3 Milestones

4.4 Resource Requirements

4.5 Risk Assessment

Chapter 5

Experimental Results

5.1 Evaluation Plan

Chapter 6

Conclusions

6.1 Summary

6.2 Expected Deliverables

6.3 Future Works

Bibliography

- [1] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, Philip Li, Akul Gupta, Adarsh Danda, Richard Fang, Conner Jensen, Eric Ihli, Jason Benn, Jet Geronimo, Avi Dhir, Sudhit Rao, Kaicheng Yu, Twm Stone, and Daniel Kang. CVE-Bench: A benchmark for AI agents’ ability to exploit real-world web application vulnerabilities. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025.
- [2] Ruozhao Yang, Mingfei Cheng, Gelei Deng, Tianwei Zhang, Junjie Wang, and Xiaofei Xie. PentestEval: Benchmarking LLM-based penetration testing with modular and stage-level design. Preprint, 2025.
- [3] Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. LLM agents can autonomously hack websites. arXiv preprint arXiv:2402.06664, 2024.
- [4] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. LLM agents can autonomously exploit one-day vulnerabilities. arXiv preprint arXiv:2404.08144, 2024. v2, cs.CR.
- [5] Yuxuan Zhu, Antony Kellermann, Akul Gupta, Philip Li, Richard Fang, Rohan Bindu, and Daniel Kang. Teams of LLM agents can exploit zero-day vulnerabilities. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026. arXiv:2406.01637.
- [6] Isaac David and Arthur Gervais. Multi-agent penetration testing AI for the web. arXiv preprint arXiv:2508.20816, 2025.

-
- [7] Lingzhi Wang, Xinyi Shi, Ziyu Li, Yi Jiang, Shiyu Tan, Yuhao Jiang, Junjie Cheng, Wenyuan Chen, Xiangmin Shen, Zhenyuan Li, and Yan Chen. Automated penetration testing with LLM agents and classical planning. arXiv preprint arXiv:2512.11143, 2025.
 - [8] Xiaoxue Ren, Penghao Jiang, Kaixin Li, Zhiyong Huang, Xiaoning Du, Jiaojiao Jiang, Zhenchang Xing, Jiamou Sun, and Terry Yue Zhuo. HackWorld: Evaluating computer-use agents on exploiting web application vulnerabilities. arXiv preprint arXiv:2510.12200, 2025.
 - [9] Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Jasper, Pura Peetathawatchai, Ari Glenn, Vikram Sivashankar, Daniel Zamoshchin, Leo Glikbarg, Derek Askaryar, Mike Yang, Teddy Zhang, Rishi Alluri, Nathan Tran, Rinnara Sangpisit, Polycarpos Yiorkadjis, Kenny Osele, Gautham Raghupathi, Dan Boneh, Daniel E. Ho, and Percy Liang. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.
 - [10] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*, pages 847–864, Philadelphia, PA, 2024. USENIX Association.
 - [11] Shaolun Liu, Sina Marefat, Omar Tsai, Yu Chen, Zecheng Deng, Jia Wang, and Mohammad A. Tayebi. PrediQL: Automated testing of GraphQL APIs with LLMs. arXiv preprint arXiv:2510.10407, 2025.
 - [12] Brian Singer, Keane Lucas, Lakshmi Adiga, Meghna Jain, Lujo Bauer, and Vyas Sekar. Incalmo: An autonomous LLM-assisted system for red teaming multi-host networks. arXiv preprint arXiv:2501.16466, 2025.
 - [13] Andy K. Zhang, Joey Ji, Celeste Menders, Riya Dulepet, Thomas Qin, Ron Y. Wang, Junrong Wu, Kyleen Liao, Jiliang Li, Jinghan Hu, Sara Hong, Nardos

- Demilew, Shivatmica Murgai, Jason Tran, Nishka Kacheria, Ethan Ho, Denis Liu, Lauren McLane, Olivia Bruvik, Dai-Rong Han, Seungwoo Kim, Akhil Vyas, Cuiyuanxiu Chen, Ryan Li, Weiran Xu, Jonathan Z. Ye, Prerit Choudhary, Siddharth M. Bhatia, Vikram Sivashankar, Yuxuan Bao, Dawn Song, Dan Boneh, Daniel E. Ho, and Percy Liang. BountyBench: Dollar impact of AI agent attackers and defenders on real-world cybersecurity systems. In *Advances in Neural Information Processing Systems (NeurIPS) 39, Track on Datasets and Benchmarks*, 2025. arXiv:2505.15216.
- [14] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. RESTler: Stateful REST API fuzzing. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 748–758. IEEE, 2019.
- [15] Andreas Happe and Jürgen Cito. Getting pwn’d by AI: Penetration testing with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023. doi: 10.1145/3611643.3613083.
- [16] Gelei Deng, Yi Liu, Yuekang Li, Ruozhao Yang, Xiaofei Xie, Jie Zhang, Han Qiu, and Tianwei Zhang. What makes a good LLM agent for real-world penetration testing? Preprint (PentestGPT V2 / TDA-EGATS), 2025.
- [17] He Kong, Die Hu, Jingguo Ge, Liangxiong Li, Tong Li, and Bingzhen Wu. VulnBot: Autonomous penetration testing for a multi-agent collaborative framework. arXiv preprint arXiv:2501.13411, 2025.
- [18] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155, 2023.
- [19] Meet Udeshi, Minghao Shao, Haoran Xi, Nanda Rani, Kimberly Milner, Venkata Sai Charan Putrevu, Brendan Dolan-Gavitt, Sandeep Kumar Shukla, Prashanth Krishnamurthy, Farshad Khorrami, Ramesh Karri, and Muhammad

- Shafique. D-CIPHER: Dynamic collaborative intelligent multi-agent system with planner and heterogeneous executors for offensive security. arXiv preprint arXiv:2502.10931, 2025.
- [20] Andreas Happe and Jürgen Cito. Can LLMs hack enterprise networks? arXiv preprint arXiv:2502.04227, 2025.
- [21] Akshat Singh Jaswal and Ashish Baghel. AWE: Adaptive agents for dynamic web penetration testing. LAST-X Workshop, Network and Distributed System Security (NDSS) Symposium; arXiv preprint arXiv:2603.00960, 2026.
- [22] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023.
- [23] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)* 36, 2023.

Appendix A: RedGrid Reference Material

This appendix provides full specifications for the REDGRID system that would exceed the length constraints of the main chapters, including the complete VDG node schema, benchmark suite details, UCB hyperparameter search configuration, and ablation condition pseudocode.

.1 Full VDG Node Schema

Table 1 provides the complete field listing for a VDG node as defined in Chapter 3. Every field is populated by the Team Manager via `VDG_AddNode`; no Specialist writes directly to the VDG.

TABLE 1: Complete VDG Node Schema

Field	Type	Description
<code>weakness_id</code>	<code>str</code>	Unique identifier (e.g. <code>sqli-001</code>)
<code>vuln_class</code>	<code>enum</code>	<code>SQLi</code> <code>XSS</code> <code>CSRF</code> <code>SSTI</code> <code>LFI</code> <code>AuthBypass</code> <code>IDOR</code> <code>RCE</code> <code>GraphQL-Injection</code> <code>LateralMove</code>
<code>attack_intent</code>	<code>str</code>	Natural language description of the goal

Field	Type	Description
<code>prerequisites</code>	<code>List[str]</code>	<code>weakness_ids</code> that must reach <code>EXPLOITED</code> before this node is eligible
<code>enables</code>	<code>List[str]</code>	<code>weakness_ids</code> that become <code>ELIGIBLE</code> once this node is <code>EXPLOITED</code>
<code>status</code>	<code>enum</code>	<code>ELIGIBLE</code> <code>IN_PROGRESS</code> <code>EXPLOITED</code> <code>INFEASIBLE</code> <code>BLOCKED</code>
<code>w</code>	<code>float</code>	Cumulative UCB reward (sum of outcome scores)
<code>n</code>	<code>int</code>	Times this node has been selected (initialised to 1 to avoid division by zero)
<code>phi</code>	<code>float</code>	Promise score $\phi \in [0, 1]$: LLM-assessed exploitability likelihood
<code>delta</code>	<code>float</code>	Task difficulty index $\delta \in [0, 1]$; 1 = maximally hard
<code>E_ord</code>	<code>int</code>	Ordinal evidence score $E_{\text{ord}} \in \{0, 1, 2, 3, 4, 5\}$ (see Table 2)
<code>epss_prior</code>	<code>float</code>	EPSS score $\in [0, 1]$ from NVD/FIRST API; default 0.05 if no CVE match
<code>context_load</code>	<code>float</code>	Estimated Specialist context cost $C \in [0, 1]$
<code>estimated_cost</code>	<code>float</code>	Estimated token cost for this node's exploitation
<code>retry_count</code>	<code>int</code>	Number of Specialist invocations on this node
<code>max_retries</code>	<code>int</code>	Default 3; on exhaustion $\rightarrow$ <code>INFEASIBLE</code>
<code>path_score</code>	<code>float</code>	Score of best feasible path this node participates in

Field	Type	Description
<code>source_el_nodes</code>	<code>List[str]</code>	EL node IDs that seeded this VDG node
<code>last_updated</code>	<code>timestamp</code>	Timestamp of last field mutation

.2 Ordinal Evidence Score (E_{ord}) Reference

Table 2 is the full rubric used by the Evaluation Agent to assign E_{ord} after every Specialist task. The value is parsed deterministically from constrained JSON output — not inferred from free-form text.

TABLE 2: Ordinal Evidence Score (E_{ord}) Rubric

E_{ord}	Label	Criteria
0	Unseen	Node not yet acted on
1	Tool ran / nothing	Tool executed; target responded normally; no anomaly detected
2	Weak signal	Anomalous response (different status code, timing difference) but ambiguous
3	Clear indication	Behavioural evidence consistent with vulnerability (e.g. SQL error message, reflected input) but not yet controlled
4	Confirmed behaviour	Controlled behaviour demonstrated (e.g. parameterised timing differential, payload executing in non-live context)
5	Oracle-confirmed	Per-surface oracle confirms successful exploitation

.3 Benchmark Suite Summary

Table 3 summarises all eight benchmark tiers used in REDGRID’s evaluation. No benchmark was constructed for this work; every tier references a published, reusable target set.

TABLE 3: Full REDGRID Benchmark Suite

Tier	Benchmark	Surface	Size	Role
0	Fang et al. sandbox	Web	15 CVEs	CI regression flow
0b	HPTSA 14-CVE suite	Web (0-day)	14 CVEs	Zero-day mode validation
1	PENTESTEVAL	Web	12 / 346	Stage-level diagnosis UCB tuning
2	CVE-BENCH	Web	40 CVEs	Primary metric (pass@1, pass@5)
2b	MAPTA / HackWorld / Cybench	Web	~180	Cross-benchmark generalisation
3	PREDIQL	GraphQL	6 APIs	Schema-coverage baselines
4	MHBENCH	Multi-host	40 envs	Host-compromise credential-theft
5	BOUNTYBENCH	Web (prod.)	25 systems	Dollar-value cost-per-exploit
6	PentestGPT set + HTB S8	Web	18 machines	Live-competition validation

.4 UCB Hyperparameter Search Grid

The UCB formula (Chapter 3) has seven tunable parameters. Table 4 documents the search ranges and defaults. Grid search is performed on Tier 1 (PENTESTEVAL) only, holding out the CVE-BENCH split.

Results must be reported with sensitivity analysis: $\pm 10\%$ perturbation from each optimal value while holding others fixed.

TABLE 4: UCB Hyperparameter Search Grid

Parameter	Meaning	Search Range	Default
C_{expl}	Exploration constant	$\{0.5, 1.0, 1.414, 2.0\}$	1.414
α	Weight for promise ϕ	$[0.1, 0.5]$ step 0.1	0.3
β	Weight for difficulty $(1 - \delta)$	$[0.1, 0.5]$ step 0.1	0.2
γ	Weight for $E_{\text{ord}}/5$	$[0.1, 0.5]$ step 0.1	0.4
κ	Context-load penalty	$[0.05, 0.2]$ step 0.05	0.1
λ	EPSS prior weight	$[0.05, 0.3]$ step 0.05	0.15
μ	Cost-awareness penalty	$[0.05, 0.2]$ step 0.05	0.1

.5 Ablation Condition A1 — Precise Pseudocode

This section provides the exact algorithmic distinction between Ablation A1 conditions (b) and (c), which is critical for interpreting whether the unified VDG adds value over a stacked approach.

Condition (b) — UCB with dependency-constrained eligible set (pre-filter):

```

eligible = [v for v in VDG.nodes
            if v.status == ELIGIBLE
            and all_prerequisites_satisfied(v)]
scores   = {v: UCB_score(v, N=len(eligible))
            for v in eligible}
selected = max(eligible, key=lambda v: scores[v])

```

Condition (c) — Stacked: UCB over all nodes, post-filter by dependency (the key discriminant):

```

all_nodes = VDG.all_unattempted_nodes()
scores    = {v: UCB_score(v, N=len(all_nodes))
            for v in all_nodes}
ranked    = sorted(all_nodes,
                  key=lambda v: scores[v],

```

```

        reverse=True)
filtered  = [v for v in ranked
            if all_prerequisites_satisfied(v)]
selected  = filtered[0]

```

If condition (d) (full VDG with path scoring) outperforms condition (c), the unified graph structure has value beyond dependency filtering alone. If (d) $\approx$ (c), the contribution downgrades to dependency-aware UCB filtering — still a contribution, but a narrower claim. This gate must be reported honestly.

.6 Prior Work Gap Summary

Table 5 consolidates the gap analysis from Chapter 2 into a compact reference table.

TABLE 5: Prior Work Gaps and REDGRID Fixes

Gap	Papers exhibiting	RedGrid fix
Flat dispatch, no prerequisite modelling	HPTSA, MAPTA, T-Agent	VDG dependency-constrained frontier
Dependency planning on pre-curated sets only	PENTESTEVAL SMP, CHECKMATE	Dynamic VDG_AddNode from Specialist discovery
Node-level scoring misses multi-step chains	EGATS	Path scoring: product of node scores $\times$ impact / cost
No failure propagation	All 29 surveyed papers	BLOCKED propagation + frontier recomputation
Exploration failure unaddressed architecturally	CVE-BENCH (diagnostic)	Full-surface Recon + dependency-constrained frontier
Session/multi-turn state loss	Fang et al., Pentest-GPT	Auth/Session Specialist + Session Persistence Service

Gap	Papers exhibiting	RedGrid fix
Context pollution from rolling history	PentestGPT, D-CIPHER, VulnBot	Fresh context per Specialist invocation
Long-session context inflation	PentestGPT Finding 4	FullCompact: EL+AL-based reconstruction at 85%
Raw LLM confidence overconfident in UCB	EGATS	E_{ord} ordinal scale (0–5) replaces raw confidence
Single-attempt validation: false positives	All single-agent systems	Diagnosis-Adapt-Cap loop (up to 3 retries)
Per-mission failure repetition	All fresh-context designs	Episodic Failure Memory (4th FAISS tier)
No cross-mission skill promotion	AWE, PrediQL, VulnBot	3-tier memory + oracle-gated skill promotion
Single benchmark surface evaluated per paper	All 29 papers	Web + GraphQL + multi-host under one orchestrator